

第一部分 编译原理概念题

第一章 Introduction

1.（整体结构 + 分工 + 目的）从整体角度描述一个典型编译器的基本组成结构。分别说明编译器前端和后端各包含哪些主要阶段，这些阶段的职责是什么？为什么要将编译器划分为前端和后端？

一个典型的编译器通常由前端和后端两大部分组成，其输入是源程序，输出是目标机器代码。

编译器的**前端**属于分析部分，主要包括词法分析、语法分析、语义分析以及中间代码生成。前端的主要职责是对源程序进行分析，检查程序是否符合语言定义的语法和语义规则，并将源程序转换为一种中间表示，同时收集程序中变量、过程等相关信息并存储在符号表中。

编译器的**后端**属于合成部分，主要包括机器无关代码优化、代码生成以及机器相关代码优化。后端的主要职责是根据前端生成的中间表示和符号表信息，构造语义等价的目标程序，并在此过程中尽可能提高程序的执行效率。

将编译器划分为前端和后端可以实现模块化设计，使前端只依赖于源语言、后端只依赖于目标机器，从而提高编译器的可移植性和可维护性。

2.（阶段职责对比型，老师很爱）请说明词法分析、语法分析和语义分析这三个阶段在编译器中的作用。它们分别解决什么问题？这三个阶段之间有什么关系？

词法分析、语法分析和语义分析是编译器前端的三个核心阶段，它们逐层完成对源程序的分析。

词法分析的作用是将源程序的字符流分解为词法单元序列，它解决的是“字符如何组成语言中的基本符号”的问题。词法分析器输出的词法单元是语法分析的输入。

语法分析的作用是根据语言的文法规则，对词法单元序列进行结构分析，构造反映程序语法结构的中间表示，通常是语法树。语法分析解决的是“程序是否具有合法结构”的问题。

语义分析的作用是检查程序结构是否具有正确的含义，它利用语法树和符号表中的信息，进行类型检查、名字绑定等工作。语义分析解决的是“结构正确的程序是否语义合理”的问题。

这三个阶段是逐步递进的，前一阶段的输出是后一阶段的输入。

3.（token / lexeme + 符号表，综合题）什么是词素（lexeme）和词法单元（token）？词法单元通常以什么形式传递给语法分析器？词法单元中的属性值有什么作用？

词素是源程序中实际出现的字符序列，是程序设计语言中最基本的句法单位，例如一个标识符名或一个运算符符号。

词法单元是对一类词素的抽象表示，用来表示具有相同语法功能的一组词汇。

在编译器中，词法单元通常以 `<token 名称, 属性值>` 的形式传递给语法分析器。其中，token 名称用于语法分析阶段进行语法结构判断。

属性值通常指向符号表中的一个条目，该条目中保存了与该词素相关的语义信息，如类型、存储位置等。这些信息将在语义分析和代码生成阶段中被进一步使用。

4.（语义分析 + “为什么只能有限”，高度像你老师）什么是语义分析？语义分析在编译器中的主要作用

是什么？请结合编程语言的特点说明，为什么编译器只能进行有限的语义分析。

语义分析是编译器中用于检查程序含义是否正确的阶段。与语法分析不同，语义分析关注的是程序在执行时应当做什么，而不仅仅是程序的结构是否正确。

语义分析器主要利用语法树和符号表中的信息，检查程序是否满足语言定义的语义规则，例如类型是否匹配、变量是否正确声明和使用，并为后续的中间代码生成收集类型相关的信息。

由于程序的实际含义涉及运行时行为，理解完整语义往往需要执行程序，因此在编译阶段进行完整语义分析是非常困难的。编译器通常只能执行有限的语义分析，用于捕捉明显的语义不一致，而语言本身通过严格规则来减少歧义。

5.（类型检查专门拎出来，容易考）什么是类型检查？为什么说类型检查是语义分析中的一个重要组成部分？请结合实例说明。

类型检查是语义分析阶段的一项重要任务，其主要目的是检查程序中各个运算符的操作数是否具有匹配的类型。

通过类型检查，编译器可以在编译阶段发现潜在的错误，防止类型不匹配的操作在运行时产生不可预期的结果。例如，许多语言要求数组索引必须是整数，如果数组访问使用了浮点数作为索引，编译器应当报告错误。

因此，类型检查对于保证程序的正确性和安全性具有重要意义。

6.（IR + 优化 + 后端承接）什么是中间表示（IR）？编译器为什么需要中间表示？中间表示在前端和后端之间扮演什么角色？

中间表示是源程序在编译过程中形成的一种中间形式，位于源语言和目标机器语言之间。语义分析完成后，编译器通常将程序转换为中间表示，例如三地址码。

使用中间表示可以将编译器的前端和后端解耦，使前端只需关注源语言的分析，而后端只需关注目标机器的代码生成。

中间表示还便于进行机器无关的代码优化，并支持将同一源程序映射到不同的目标机器。

因此，中间表示在前端和后端之间起着桥梁作用。

7.（符号表，多阶段 + 内容）什么是符号表？符号表通常记录哪些信息？请说明符号表在编译器前端和后端中的作用。

符号表是编译器中用于记录程序中名字及其属性的数据结构。

符号表通常记录变量和过程的名称、类型、作用域、存储分配信息，以及过程的参数数量、参数类型、参数传递方式和返回类型等。

符号表主要由编译器前端建立和维护，在语义分析阶段用于类型检查和名字绑定；在后端阶段，符号表中的存储和类型信息将用于代码生成和存储分配。

8.（编译器 vs 解释器，经典收官题）请比较编译器和解释器在程序执行方式和分析过程上的主要区别。

编译器在程序执行之前，将整个源程序翻译成目标机器代码，程序只有在成功编译后才能运行。编译器通常会对程序进行较深入的分析，并可以执行多种优化。

解释器则不生成独立的目标程序，而是在运行时逐条解析并执行源代码。解释器通常花费较少时间分析源代码，但一旦遇到错误便会停止执行。

第二章 Lexical Analysis（词法分析）

1.（总体职责 + 与语法分析的关系）什么是词法分析？词法分析器在编译器中承担哪些主要任务？词法分析器与语法分析器是如何交互的？

词法分析是编译器的第一个阶段，其主要任务是从源程序的字符流中识别出有意义的词素，并将它们组织成词法单元序列，作为语法分析器的输入。

词法分析器的核心任务包括：

第一，将输入字符流划分为一个个词素，并为每个词素生成相应的词法单元；

第二，在识别标识符时与符号表进行交互，将新标识符加入符号表或查询已有条目；

第三，去除源程序中的空白符和注释；

第四，为错误处理提供支持，例如记录行号信息。

在实际实现中，语法分析器通常通过调用词法分析器提供的接口（如 `getNextToken`）来获取下一个词法单元，二者通过这种“请求—返回”的方式协同工作。

2.（为什么要分离）为什么编译器通常将词法分析和语法分析分成两个独立阶段？请从设计、效率和可移植性角度说明原因。

将词法分析和语法分析分离是编译器设计中的一个重要原则。

首先，从设计角度看，分离词法分析可以简化语法分析器的设计。如果语法分析器还需要处理空白符和注释，其文法将变得更加复杂，不利于语言结构的清晰描述。

其次，从效率角度看，词法分析可以采用专门的高速扫描技术和缓冲机制，从而显著提高编译速度。

最后，从可移植性角度看，与输入设备相关的处理（如换行符表示）可以集中在词法分析器中，使编译器更容易适配不同系统环境。

3.（核心概念对比）请解释词素（lexeme）、词法单元（token）、模式（pattern）三者之间的关系，并说明它们在词法分析中的作用。

词素是源程序中实际出现的字符序列，是与某个词法单元模式相匹配的具体实例。

词法单元是对一类具有相同语法功能的词素的抽象表示，通常由词法单元名称和可选的属性值组成。

模式用于描述某一类词法单元对应的词素形式，通常使用正则表达式来表示。

在词法分析过程中，模式用于匹配输入字符流，识别出具体的词素；词素是模式匹配的结果；而词法单元则是词法分析器输出给语法分析器的形式。

4.（属性值 + 符号表）词法单元的属性值有什么作用？为什么标识符类词法单元通常需要属性值，而关键字和运算符往往不需要？

词法单元的属性值用于向编译器后续阶段传递与具体词素相关的附加信息。

对于标识符而言，其名称、类型、作用域、存储位置等信息都需要在后续阶段使用，因此属性值通常是指向符号表中对应条目的指针。

而对于关键字、运算符和标点符号，其语义已经由词法单元名称唯一确定，不需要额外信息，因此通常不需要属性值。

5.（错误处理视角）词法分析器在错误处理方面能做哪些工作？哪些错误通常不能由词法分析器单独发现？

词法分析器可以处理一些低层次的错误，例如无法匹配任何词法单元模式的字符序列。常见的错误恢复

策略是恐慌模式恢复，即不断丢弃输入字符，直到可以重新识别词法单元为止。

然而，词法分析器通常无法判断某个合法词素在语义上是否正确。例如，将 `fi` 误写成 `if` 时，`fi` 仍然可能是合法的标识符，这类错误通常需要由语法分析或语义分析阶段来处理。

6.（正则表达式的角色）为什么正则表达式非常适合用于描述词法单元的模式？正则表达式能描述哪些类型的语言？

正则表达式能够简洁、精确地描述字符模式，并且支持高效的自动机实现，因此非常适合用于描述词法单元的模式。

正则表达式所能描述的语言称为正则语言，这类语言恰好是有限自动机可以识别的语言。大多数程序设计语言中的词法结构（如标识符、常量、关键字）都属于正则语言。

7.（语言与字符串，概念题常考）什么是字母表、字符串和语言？语言的并、连接和闭包运算各自表示什么含义？

字母表是一个有限符号集合。字符串是由字母表中符号组成的有限序列。语言是定义在某一字母表上的字符串集合。

语言的并运算表示属于任一语言的字符串集合；

连接运算表示从两个语言中各取一个字符串并进行连接后形成的集合；

闭包运算表示对某语言进行零次或多次连接得到的字符串集合。

这些运算构成了正则表达式的理论基础。

8.（自动机引入动机）什么是有限自动机？有限自动机在词法分析中的作用是什么？

有限自动机是一种具有有限个状态的抽象计算模型，用于判断输入字符串是否属于某一语言。

在词法分析中，有限自动机用于识别与词法单元模式相匹配的输入字符串。词法分析器通过模拟自动机的状态转移过程，完成对词素的识别。

9.（NFA vs DFA，对比型）请比较非确定性有限自动机（NFA）和确定性有限自动机（DFA）的主要区别，并说明为什么词法分析器通常使用 DFA。

NFA 允许在同一状态下对同一输入符号存在多条转移，并允许空转换；而 DFA 对于任一状态和输入符号，都有且仅有一条确定的转移。

虽然 NFA 和 DFA 能识别的语言集合相同，但 DFA 的执行过程更加简单高效，因此词法分析器通常使用 DFA 来实现模式匹配。

10.（转换过程，老师很爱问）为什么需要将正则表达式或 NFA 转换为 DFA？这一转换的基本思想是什么？

正则表达式和 NFA 更适合描述模式，但不便于直接高效执行；而 DFA 的执行效率高，适合实际实现。

将 NFA 转换为 DFA 的基本思想是子集构造法，即让 DFA 的每个状态对应 NFA 的一个状态集合，从而将非确定性选择转化为确定性转移。

11.（冲突与最长匹配）当多个模式可以匹配输入的同一前缀时，词法分析器通常如何解决冲突？

词法分析器通常遵循两条规则：

第一，优先选择能够匹配的最长输入前缀；

第二，如果最长前缀同时匹配多个模式，则选择在词法规则中先定义的模式。

这两条规则保证了词法分析结果的不确定性。

12.（Lex 工具，总结性题目）Lex 工具的作用是什么？Lex 程序的基本结构由哪几部分组成？

Lex 是一种自动生成词法分析器的工具。它允许用户使用正则表达式描述词法单元模式，并自动生成对应的 C 程序。

一个 Lex 程序通常由三部分组成：声明部分、规则部分和辅助函数部分。声明部分用于定义常量和正则定义；规则部分由“模式—动作”对组成；辅助函数部分包含用户自定义的支持代码。

第三章 Syntax Analysis（语法分析）

1.（语法分析总体定位）什么是语法分析？语法分析器在编译器中的角色是什么？语法分析器的输入和输出分别是什么？

语法分析是编译器前端的重要阶段，其主要任务是根据程序设计语言的语法规则，检查由词法分析器生成的词法单元序列是否能够由该语言的文法生成。

语法分析器的输入是词法分析器提供的 token 流，输出通常是反映程序语法结构的中间表示，最常见的形式是语法分析树。

对于语法正确的程序，语法分析器能够构造出相应的语法结构；对于存在语法错误的程序，语法分析器应当能够检测并以合理的方式报告错误。

2.（CFG 基础，必考）什么是上下文无关文法（CFG）？CFG 由哪些部分组成？为什么 CFG 非常适合用于描述程序设计语言的语法？

上下文无关文法是一种用于描述语言语法结构的形式化工具，广泛用于程序设计语言的语法定义。

一个 CFG 由四个部分组成：终结符集合、非终结符集合、产生式集合以及开始符号。产生式的形式为 **头部** → **体部**，其中头部必须是一个非终结符，体部由零个或多个终结符或非终结符组成。

CFG 能够自然地描述程序语言中的嵌套结构和递归结构，例如表达式、语句块等，因此比正则表达式具有更强的表达能力，非常适合用于描述编程语言的语法。

3.（推导 + 句型 + 句子）什么是推导？什么是文法的句型和句子？三者之间有什么关系？

推导是指从文法的开始符号出发，反复使用产生式替换非终结符，逐步得到字符串的过程。

如果从开始符号可以推导出某个字符串，则该字符串称为文法的句型。句型中可以同时包含终结符和非终结符，也可以是空串。

如果某个句型中不再包含任何非终结符，则该字符串称为文法的句子。

文法所生成的语言，正是它能够生成的所有句子的集合。

4.（最左 / 最右推导 + 语法树）什么是最左推导和最右推导？它们与语法分析树之间有什么关系？

在推导过程中，每一步都需要选择一个非终结符进行替换。

最左推导是指在每一步中，总是选择当前句型中最左边的非终结符进行替换；

最右推导则总是选择最右边的非终结符进行替换。

语法分析树是推导过程的一种图形化表示。

虽然同一棵语法分析树可能对应多种不同的推导序列，但最左推导和最右推导与语法分析树之间是一一对应的关系，因此在语法分析中具有重要意义。

5.（二义性，老师特别爱）什么是文法的二义性？二义性会带来什么问题？程序设计语言通常如何处理二义性？

如果一个文法能够为同一个句子生成多棵不同的语法分析树，则称该文法是二义的。

文法二义性会导致程序的含义不明确，使得同一程序可能对应多种不同的解释，从而给编译器的语义分析和代码生成带来困难。

因此，程序设计语言的语法通常要求是无二义的，或者通过引入消歧规则（如运算符优先级、结合性规则）来消除不需要的语法分析树。

6. (CFG vs 正则表达式, 对比型) 为什么 CFG 比正则表达式具有更强的表达能力？请结合实例说明。

CFG 比正则表达式具有更强的表达能力，因为 CFG 可以描述具有嵌套和递归结构的语言，而正则表达式只能描述正则语言。

例如，语言

$(L = \{ a^n b^n \mid n > 0 \})$

可以用 CFG 描述，但无法用正则表达式或 DFA 描述。这类语言在程序设计语言中非常常见，例如成对匹配的括号结构，因此需要使用 CFG 来描述。

7. (文法改造, 连续考点) 为什么在语法分析之前需要对文法进行改造？常见的文法改造包括哪些？

在实际语法分析之前，原始文法往往不适合直接用于构造语法分析器，因此需要进行改造。

常见的文法改造包括：

- 消除二义性
- 消除左递归
- 提取左公因子

这些改造可以使文法更适合自顶向下或自底向上分析，从而避免回溯、无限递归等问题。

8. (左递归, 必考) 什么是左递归？为什么自顶向下语法分析不能直接处理左递归文法？

如果文法中存在某个非终结符 A ，使得 A 可以推导出以 A 开头的串，即

$(A \rightarrow^+ A\alpha)$ ，

则称该文法存在左递归。

在自顶向下语法分析中，左递归会导致递归下降分析器不断调用自身而无法消耗输入符号，从而陷入无限递归，因此在使用自顶向下分析之前必须消除左递归。

9. (左公因子提取, 决策推迟) 什么是左公因子提取？为什么左公因子提取有助于语法分析决策？

左公因子提取是指当多个产生式具有公共前缀时，将公共前缀提取出来，从而推迟产生式选择决策。

通过左公因子提取，语法分析器可以在读取更多输入符号后再决定使用哪一个产生式，这对于构造无回溯的预测分析器尤为重要。

10. (自顶向下 vs 自底向上) 请比较自顶向下语法分析和自底向上语法分析的基本思想和主要差异。

自顶向下语法分析从文法的开始符号出发，试图通过一系列推导生成输入字符串，相当于构造最左推导。

自底向上语法分析从输入字符串出发，不断进行归约，最终归约到开始符号，相当于反向构造最右推导。

自底向上分析通常比自顶向下分析更强大，能够处理更一般的文法，但实现也更加复杂。

11. (FIRST / FOLLOW, 经典大题) 什么是 FIRST 集和 FOLLOW 集？它们在语法分析中各自起什么作用？

FIRST 集表示从某个语法符号或符号串推导得到的字符串中可能出现的第一个终结符集合。

FOLLOW 集表示在某些句型中可以紧跟在某非终结符后面的终结符集合。

FIRST 和 FOLLOW 集主要用于构造预测分析表，帮助语法分析器在自顶向下分析过程中，根据当前输入符号选择正确的产生式。

12. (LL(1), 条件型) 什么是 LL(1) 文法? LL(1) 文法需要满足哪些条件?

LL(1) 文法是一类可以用一个向前看符号进行无回溯自顶向下语法分析的文法。

LL(1) 文法要求：

对于同一非终结符的不同产生式，其 FIRST 集不相交；

如果某个产生式可以推导出空串，则该产生式的 FIRST 集与该非终结符的 FOLLOW 集也必须不相交。

这些条件保证了分析器在每一步都能唯一确定使用的产生式。

13. (移入-归约分析) 什么是移入-归约语法分析? 移入和归约分别表示什么操作?

移入-归约分析是一种通用的自底向上语法分析技术。

移入操作是将下一个输入符号移入栈中；

归约操作是将栈顶与某个产生式体匹配的符号串替换为该产生式的头部。

通过反复执行移入和归约，最终将输入字符串归约为开始符号。

14. (句柄, 理解型) 什么是句柄? 为什么在移入-归约分析中句柄总是出现在栈顶?

句柄是与某个产生式体相匹配的子串，对句柄进行归约相当于最右推导中的一个反向步骤。

在移入-归约分析过程中，语法分析器总是通过移入操作不断扩展栈顶，直到可以在栈顶识别出一个句柄，因此不需要在栈中间寻找句柄，这保证了算法的高效性。

15. (冲突, 老师非常爱考) 什么是移入/归约冲突和归约/归约冲突? 它们通常由什么原因引起?

移入/归约冲突是指在某个状态下，语法分析器既可以选择移入，也可以选择归约；

归约/归约冲突是指分析器无法在多个可能的归约产生式中作出选择。

这类冲突通常由文法的二义性或文法不满足相应分析方法的条件引起。

16. (LR 总结型) 为什么说 LR 语法分析器比 LL 语法分析器更强大?

LR 语法分析器能够处理的文法集合比 LL 文法更大，几乎可以为所有实际编程语言构造语法分析器。

同时，LR 语法分析是无回溯的移入-归约分析方法，具有较强的错误检测能力，因此在实际编译器中应用非常广泛。

17. (错误恢复, 概念题) 语法分析中的错误恢复主要有哪些策略? 各自的基本思想是什么?

常见的错误恢复策略包括恐慌模式恢复和短语层次恢复。

恐慌模式恢复通过丢弃输入符号直到遇到同步词法单元为止，从而跳过错误部分继续分析；

短语层次恢复则尝试对栈或输入进行局部修改，并向程序员给出更精确的错误提示。

18. (Yacc, 工具型总结) Yacc 工具的作用是什么? 一个 Yacc 程序通常由哪几部分组成?

Yacc 是一种语法分析器生成工具，用于根据上下文无关文法自动构造语法分析器。

一个 Yacc 程序通常由三部分组成：

声明部分、翻译规则部分（产生式与语义动作）以及辅助 C 语言程序部分。

Yacc 通常与 Lex 配合使用，共同完成编译器前端的构造。

第四章 Syntax-Directed Translation（语法制导翻译）

1.（总体定位 + 与前后阶段关系）什么是语法制导翻译？它在编译器中的位置是什么？它与词法分析、语法分析和中间代码生成之间有什么关系？

语法制导翻译是指在语法分析过程中，按照语法结构并结合语义规则，对源程序进行语义处理和翻译的一种方法。

在编译器中，语法制导翻译通常位于语法分析之后、代码生成之前，是连接语法分析和中间代码生成的重要桥梁。

它以语法分析得到的解析结构为基础，通过属性和语义规则描述如何计算表达式的值、生成中间代码或构造语法树。

语法制导翻译并不取代词法分析和语法分析，而是在它们提供的结构基础上，完成语义信息的计算和翻译工作。

2.（SDD 核心定义题）什么是语法制导定义（SDD）？SDD 由哪些基本要素组成？SDD 主要解决什么问题？

语法制导定义（SDD）是一种在上下文无关文法基础上引入属性和语义规则的形式化描述方法。

SDD 的基本要素包括：

- 与每个语法符号相关联的一组属性；
- 与每个产生式相关联的一组语义规则，用于描述如何计算这些属性。

SDD 的主要作用是刻画语法结构的语义含义，它关注的是**“属性值如何在语法树上计算”**，而不是具体如何生成代码。

因此，SDD 常用于描述表达式求值、类型计算等语义问题。

3.（属性分类，必考）什么是综合属性和继承属性？它们各自的依赖方向是什么？在语法制导翻译中各有什么作用？

综合属性是指某个节点上的属性值仅由该节点的子节点以及该节点自身的属性值决定的属性，其信息流方向是从子节点向父节点传递。

继承属性是指某个节点上的属性值由该节点的父节点或其兄弟节点的性质值决定的属性，其信息流方向通常是从父节点或左兄弟节点向当前节点传递。

综合属性适合用于描述自底向上的语义计算，而继承属性在处理与上下文相关的语义问题（如类型传递、作用域信息传播）时非常有用。

4.（属性求值顺序 + 依赖图）为什么在 SDD 中需要考虑属性的计算顺序？什么是依赖图？依赖图在属性求值中起什么作用？

在 SDD 中，属性之间可能存在依赖关系，因此必须保证在计算某个属性之前，其依赖的属性已经被计算。

依赖图是一种用于描述语法树中属性实例之间依赖关系的有向图。

图中的节点表示属性实例，有向边表示“一个属性的计算依赖于另一个属性”。

如果依赖图中存在环，则无法找到合法的属性计算顺序；如果依赖图是有向无环图，则可以通过拓扑排

序确定一种合法的属性求值顺序。

5. (良定义 SDD) 什么是良定义的语法制导定义？为什么判断一个 SDD 是否良定义在实践中是困难的？

如果一个 SDD 对于任意解析树，其对应的属性依赖图都是有向无环图，则称该 SDD 是良定义的。判断一个 SDD 是否良定义在实践中是困难的，因为不同输入可能产生不同的解析树，而解析树的结构直接影响属性之间的依赖关系。

因此，在实际编译器设计中，通常使用一些受限的 SDD 类型来保证属性求值的可行性。

6. (S 属性 SDD, 必考) 什么是 S 属性的语法制导定义？S 属性 SDD 有什么特点？为什么它易于实现？

如果一个 SDD 中的所有属性都是综合属性，则称该 SDD 为 S 属性 SDD。

由于综合属性的依赖关系只从子节点指向父节点，因此 S 属性 SDD 的依赖图中不可能出现环。

这使得属性可以在解析树的一次自下而上的遍历中完成计算，通常对应后序遍历。

因此，S 属性 SDD 非常适合在自底向上的语法分析过程中实现，易于与 LR 语法分析器结合。

7. (L 属性 SDD, 必考) 什么是 L 属性的语法制导定义？L 属性 SDD 对继承属性有哪些限制？为什么这些限制是必要的？

L 属性 SDD 是允许使用继承属性的一类语法制导定义，但对继承属性的依赖关系施加了严格限制。

在产生式

$$A \rightarrow X_1 X_2 \dots X_n$$

中， X_j 的继承属性只能依赖于：

- A 的继承属性；
- X_1 到 X_{j-1} 的属性。

这些限制保证了属性依赖关系不会从右向左传播，从而确保在从左到右的遍历过程中可以正确计算属性值。

因此，L 属性 SDD 适合在自顶向下语法分析过程中实现。

8. (SDD vs SDT, 对比型) 什么是语法制导翻译方案 (SDT)？SDT 与 SDD 有什么区别和联系？

SDT 是一种在上下文无关文法中直接嵌入语义动作的翻译方案，其语义动作在语法分析过程中被立即执行。

与 SDD 相比，SDD 关注“属性如何计算”，属于较高层次的语义描述；

而 SDT 更关注“在何时执行什么动作”，通常直接用于生成中间代码或目标代码。

可以认为 SDT 是 SDD 在具体实现层面上的一种形式，二者在实际编译器中往往是交织使用的。

9. (SDT 的实现思想) 为什么说任何 SDT 都可以通过构造解析树再执行语义动作来实现？这一思想的核心是什么？

从理论上讲，任何 SDT 都可以先构造一棵解析树，然后将语义动作视为附加在解析树节点上的“虚拟节点”，再按照从左到右的深度优先遍历顺序执行这些动作。

这一思想的核心在于：

语义动作的执行顺序本质上依赖于语法结构，而解析树完整地表达了这种结构，因此可以通过遍历解析树来统一实现 SDT。

10. (SDT 与 LL / LR 的配合, 重点) 在什么条件下 SDT 可以在语法分析过程中直接执行? 为什么并非所有 SDT 都能在分析过程中执行?

当 SDT 的语义动作位置满足 LL 或 LR 语法分析的要求, 并且不会引入分析冲突时, SDT 可以在语法分析过程中直接执行。

如果语义动作的执行顺序与语法分析的推导顺序不一致, 或者会破坏分析表的构造条件, 则无法在解析过程中直接执行, 只能采用构造解析树后再执行动作的方法。

11. (LR + S 属性, 经典搭配) 为什么在 LR 语法分析中, S 属性 SDD 特别适合实现为后缀 SDT?

在 LR 语法分析中, 归约操作发生时, 产生式右部的符号已经全部出现在分析栈中, 其综合属性也已经计算完成。

因此, 如果 SDD 是 S 属性的, 就可以在归约发生时执行语义动作, 并根据栈中已有的属性值计算产生式左部的属性。

由于语义动作位于产生式末尾, 这种 SDT 被称为后缀 SDT。

12. (LL + L 属性, 经典搭配) 为什么 L 属性 SDD 适合在 LL 语法分析过程中实现? 通常采用什么方法实现?

在 LL 语法分析中, 解析过程是自顶向下、从左到右进行的, 这与 L 属性 SDD 对继承属性依赖方向的限制是相一致的。

通常通过扩展递归下降分析器来实现 L 属性 SDD:

使用函数参数传递继承属性,

使用函数返回值或全局变量表示综合属性,

从而在解析过程中同时完成语法分析和属性计算。

13. (解析树 vs 语法树, 必考) 什么是解析树? 什么是语法树? 二者在结构和用途上有什么区别?

解析树也称为具体语法树, 其内部节点表示非终结符, 完整反映了文法推导过程。

语法树是一种更抽象的表示形式, 其内部节点表示编程结构或运算符, 子节点表示操作数。

与解析树相比, 语法树去除了不必要的中间非终结符, 更接近程序的本质结构, 通常用于中间代码生成和优化。

14. (构造语法树, 综合题) 在语法制导翻译中, 如何利用 S 属性 SDD 构造表达式的语法树? 其基本思想是什么?

构造语法树时, 可以为每个语法树节点定义一个综合属性, 用于保存指向该节点的指针。

在产生式对应的语义规则中, 通过创建新节点并连接子节点, 逐步自底向上构造整棵语法树。

这种方法利用了 S 属性自下而上的特点, 非常适合在自底向上的语法分析过程中实现。

15. (总结型大题) 请从“表达能力”和“实现方式”两个角度, 总结 S 属性 SDD、L 属性 SDD、SDD 与 SDT 之间的关系。

S 属性 SDD 表达能力较弱, 但实现简单, 适合自底向上的语法分析;

L 属性 SDD 表达能力更强, 允许受限的继承属性, 适合自顶向下分析;

SDD 是对语义计算的抽象描述, 而 SDT 是对翻译过程的具体实现方案。

在实际编译器中, 通常根据底层语法分析方法选择合适的 SDD 类型, 并将其实现为相应的 SDT。

第五章 Intermediate-Code Generation（中间代码生成）

1.（整体定位 + 前后端承接）什么是中间代码生成？为什么编译器需要中间表示？中间表示在前端和后端之间起什么作用？

中间代码生成是编译器前端的重要组成部分，其主要任务是在完成语法分析和语义分析之后，将源程序翻译为一种与源语言和目标机器都相对无关的中间表示形式。

引入中间表示的主要原因在于解耦编译器的前端和后端。前端只需关注源语言的语法和语义，而后端只需关注如何将中间表示映射为目标机器代码。通过这种方式，可以用 m 个前端和 n 个后端构造 $m \times n$ 个编译器，从而显著减少实现成本。

中间表示在前端和后端之间起到桥梁作用，同时也是代码优化的重要基础。

2.（中间表示层次）编译器中为什么可能存在多种中间表示？高层中间表示和低层中间表示各自适合完成哪些任务？

在将源程序翻译为目标代码的过程中，编译器可能构造出一系列不同层次的中间表示。

高层中间表示更接近源语言，例如语法树或抽象语法树，它们清晰地反映了程序的层次结构，适合进行静态类型检查和语义分析。

低层中间表示更接近目标机器，例如包含显式控制流和跳转的三地址代码，适合进行机器相关的优化、寄存器分配和指令选择。

通过在不同阶段选择合适层次的中间表示，可以简化各阶段的处理任务。

3.（语法树 vs DAG，经典对比）什么是语法树？什么是有向无环图（DAG）？DAG 相比语法树有哪些优势？

语法树是一种树形结构表示，内部节点表示运算符，叶子节点表示操作数，它反映了程序的语法结构。

DAG 是语法树的一种推广形式，允许多个父节点共享同一个子节点，用于表示公共子表达式。

与语法树相比，DAG 能够避免重复表示相同的子表达式，从而更紧凑地表示表达式结构，并为公共子表达式消除等优化提供重要信息。

4.（三地址代码引入动机）什么是三地址代码？为什么三地址代码非常适合作为中间表示？

三地址代码是一种中间表示形式，其基本特点是每条指令最多包含一个运算符和三个地址，常见形式为

$x = y \text{ op } z$ 。

将复杂表达式分解为多条简单的三地址指令，使得程序的计算过程更加明确，便于进行代码优化和目标代码生成。此外，三地址代码与机器指令形式相近，但又不依赖具体机器，因此非常适合作为中间表示。

5.（地址的概念，必考）三地址代码中的“地址”可以有哪些形式？临时变量的作用是什么？

三地址代码中的地址可以是以下几种形式：

1. 程序中出现的名字（通过符号表条目表示）；
2. 常量；
3. 编译器生成的临时变量。

临时变量用于保存中间计算结果，使复杂表达式能够被逐步分解。临时变量在后续优化和寄存器分配阶段可以被合并或消除，因此在中间代码生成阶段大量使用临时变量是合理且必要的。

6. (三地址指令形式, 老师去年考过) 请列出常见的三地址指令形式, 并说明各自的用途。

常见的三地址指令形式包括：

- 赋值与算术运算指令 (如 $x = y + z$ 、 $x = -y$)；
- 复制指令 (如 $x = y$)；
- 无条件跳转指令 (`goto L`)；
- 条件跳转指令 (如 `if x relop y goto L`)；
- 过程调用与返回指令 (`param x`、`call p, n`、`return y`)；
- 带下标和指针的赋值指令 (如 $x = y[i]$ 、 $*x = y$)。

这些指令共同描述了程序的数据操作和控制流。

7. (四元式 / 三元式 / 间接三元式, 对比型) 什么是四元式、三元式和间接三元式? 它们各自的优缺点是什么?

四元式用 (`op`, `arg1`, `arg2`, `result`) 四个字段表示一条三地址指令, 结构清晰, 便于代码移动和优化。三元式不显式保存结果地址, 而是通过指令位置引用运算结果, 节省空间, 但当指令重排时需要更新引用。

间接三元式通过额外的指针表间接引用三元式, 使得在不修改三元式本身的情况下调整指令顺序, 结合了两者的优点。

8. (SSA, 理解型) 什么是静态单赋值形式 (SSA)? SSA 的基本思想是什么?

静态单赋值形式是一种中间表示形式, 其基本特征是: 每个变量在程序中只被赋值一次。

SSA 通过为同一变量的不同赋值引入不同版本的名字, 并使用 ϕ 函数合并来自不同控制流路径的值, 使数据依赖关系更加清晰, 从而有利于进行多种代码优化。

9. (ϕ 函数, 必考概念) ϕ 函数的作用是什么? 它解决了 SSA 中的什么问题?

ϕ 函数用于在控制流汇合点合并来自不同路径的变量定义。

当同一变量在不同控制流分支中被赋值时, ϕ 函数根据实际执行路径选择对应的值, 从而保证 SSA 形式下每个变量仍然只被赋值一次。

10. (类型检查定位) 什么是静态类型检查? 类型信息在中间代码生成中起哪些作用?

静态类型检查是在编译阶段进行的类型一致性检查, 其目的是保证运算符的操作数类型与语言定义相匹配。

类型信息在中间代码生成中用于：

- 计算存储空间大小；
- 计算数组元素地址；
- 插入必要的类型转换；
- 选择正确的运算符或指令版本。

11.（类型表达式，抽象题）什么是类型表达式？类型表达式如何用于描述复杂类型？

类型表达式是一种用结构化形式描述类型的方式。

基本类型、数组类型、记录类型和函数类型都可以用类型构造算子组合形成类型表达式。

例如，二维数组类型可以表示为 `array(2, array(3, integer))`，其结构清晰地反映了类型的层次关系。

12.（强类型 vs 弱类型）什么是强类型语言？什么是弱类型语言？二者在编译器实现上的主要区别是什么？

如果编译器能够保证其接受的程序在运行时不会发生类型错误，则称该语言是强类型的；否则称为弱类型的。

强类型语言通常在编译阶段进行严格的类型检查，而弱类型语言允许更多隐式类型转换，将部分检查推迟到运行时。

13.（类型合成与类型推导）什么是类型合成？什么是类型推导？二者有何区别？

类型合成是根据子表达式的类型确定表达式整体类型的过程；

类型推导是根据表达式的使用方式反推出其组成部分的类型。

类型合成依赖已知类型信息，而类型推导强调通过上下文约束推断类型。

14.（数组地址计算，必考）如何计算数组元素的相对地址？为什么该问题在中间代码生成中非常重要？

数组元素的相对地址通过基地址加偏移量计算，其中偏移量由下标和元素宽度共同决定。

将地址计算公式显式地反映在中间代码中，有助于后续的优化和目标代码生成，因此是中间代码生成中的关键问题。

15.（布尔表达式两种角色）布尔表达式在程序中有哪些不同用途？为什么需要采用不同的翻译方式？

布尔表达式既可以用于计算逻辑值，也可以用于控制程序的执行流程。

当布尔表达式用于控制流时，通常采用跳转代码并利用短路求值；

当用于赋值或计算时，则需要生成计算布尔值的代码。

不同用途决定了不同的翻译策略。

16.（短路求值，概念题）什么是布尔表达式的短路求值？为什么短路求值对中间代码生成是有意义的？

短路求值是指在布尔表达式的值已经确定时，不再对剩余部分进行求值。

短路求值可以减少不必要的计算，并且自然地与控制流跳转结合，因此在中间代码生成中被广泛采用。

17.（回填技术，老师很爱）什么是回填（backpatching）？它解决了中间代码生成中的什么问题？

回填是一种用于处理跳转目标尚未确定情况的技术。

在生成跳转指令时，先将跳转目标留空，并把这些指令记录在列表中；当目标地址确定后，再统一填入。

回填技术使得一趟式生成控制流代码成为可能。

18.（综合题，总结）请从“表示能力”和“实现复杂度”的角度，总结三地址代码、SSA 和语法树在中间代码生成中的作用。

语法树表示能力强，结构清晰，适合语义分析；

三地址代码结构简单，接近机器，适合优化和代码生成；

SSA 明确表达数据依赖关系，特别适合高级优化。

三者编译器中各司其职，共同支撑从源程序到目标代码的翻译过程。

第六章 Run-Time Environments（运行时环境）

1.（整体定位 + 目的）什么是运行时环境？编译器为什么需要创建和管理运行时环境？运行时环境主要解决哪些问题？

运行时环境是目标程序执行时所处的抽象环境，由编译器、操作系统和目标机器共同支持。

编译器通过构造和管理运行时环境，使得源语言中定义的各种抽象概念能够在目标机器上正确实现。

运行时环境主要解决以下问题：

- 第一，为源程序中命名的对象分配和布局存储空间；
- 第二，确定目标程序访问变量和数据时所采用的机制；
- 第三，支持过程调用、参数传递以及过程之间的链接；
- 第四，提供与操作系统、输入输出设备和其他程序交互的接口。

2.（逻辑地址空间，结构题）什么是逻辑地址空间？一个目标程序在运行时的逻辑地址空间通常如何划分？

逻辑地址空间是从程序角度看到的地址空间，每个程序值在其中都有一个逻辑地址。

操作系统负责将逻辑地址映射为物理地址，而物理地址对整个内存进行编址。

一个目标程序的逻辑地址空间通常划分为以下区域：

- 代码区（Code）：存放目标程序的指令序列；
- 静态区（Static）：存放在编译时即可确定大小和位置的数据对象，如全局变量；
- 栈区（Stack）：用于存放活动记录，支持过程调用；
- 堆区（Heap）：用于存放生命周期不确定或跨过程调用的数据对象。

3.（静态 vs 动态存储分配）什么是静态存储分配？什么是动态存储分配？二者各自适用于哪些数据对象？

如果编译器在编译时即可确定数据对象的存储位置，则称该存储分配为静态的。

全局变量和全局常量通常采用静态存储分配。

如果数据对象的存储位置只能在程序运行时确定，则称该存储分配为动态的。

局部变量、临时变量以及动态创建的对象通常采用动态存储分配。

4.（栈存储 vs 堆存储，对比型）栈存储和堆存储各自的特点是什么？它们分别适合管理哪类数据？

栈存储用于管理局部于某个过程的数据对象，其分配和回收遵循后进先出的顺序。

栈中数据的生命周期与过程调用严格一致，管理简单、效率高。

堆存储用于管理生命周期不确定、可能跨越过程调用的数据对象，例如通过 `malloc` 或 `new` 创建的对象。

堆存储更加灵活，但管理复杂，容易产生内存碎片。

5.（活动树，老师爱考“为什么”）什么是活动树？活动树与过程调用和返回之间有什么关系？

活动树是一种描述程序执行期间过程活动关系的树结构。

树中的每个结点表示一次过程活动，根结点表示主程序的活动。

活动树具有以下重要性质：

- 过程调用的顺序对应于活动树的先序遍历；
- 过程返回的顺序对应于活动树的后序遍历；
- 当前正在执行的活动以及其所有祖先活动构成当前的活跃活动集合。

这些性质为使用运行时栈管理过程调用提供了理论基础。

6.（运行时栈，结构理解题）什么是运行时栈？运行时栈与活动树之间有什么对应关系？

运行时栈是一种用于管理过程调用和返回的动态数据结构。

每个活跃的过程活动在运行时栈中对应一个活动记录。

运行时栈中活动记录的顺序，恰好对应活动树中从根结点到当前活动结点的路径。

栈顶元素始终对应当前正在执行的过程活动。

7.（活动记录，必考结构题）什么是活动记录？一个典型的活动记录通常包含哪些内容？

活动记录是在过程调用期间分配的数据结构，用于保存与该过程活动相关的运行时信息。

一个典型的活动记录通常包含：

- 局部变量和临时变量；
- 保存的机器状态（如返回地址和寄存器内容）；
- 控制链，用于指向调用者的活动记录；
- 访问链，用于访问非局部变量；
- 参数区域；
- 返回值存放位置。

8.（控制链 vs 访问链，对比型）什么是控制链？什么是访问链？二者分别用于解决什么问题？

控制链用于连接调用者和被调用者的活动记录，主要用于在过程返回时恢复调用者的运行环境。

访问链用于支持非局部变量的访问，特别是在具有嵌套作用域的语言中，通过访问链可以定位定义该变量的活动记录。

二者在作用和用途上是不同的。

9.（调用序列，过程型大题）什么是调用序列？调用序列和返回序列各自需要完成哪些主要任务？

调用序列是在过程调用发生时，用于建立新活动记录并转移控制权的一段代码。

其主要任务包括：

- 计算并传递实在参数；
- 保存返回地址和必要的机器状态；
- 调整栈指针，为被调用过程分配活动记录空间。

返回序列是在过程结束时执行的代码，其主要任务是：

- 保存返回值；
- 恢复调用者的机器状态；
- 将控制权转回调用点之后的位置。

10.（调用者 / 被调用者职责划分）在设计调用序列时，为什么需要在调用者和被调用者之间划分职责？划分原则是什么？

调用序列的代码通常被分为调用者部分和被调用者部分。

由于被调用者的代码只生成一次，而调用者的代码可能在多个调用点重复生成，因此应尽量将与被调用者自身相关的操作放在被调用者中。

划分原则是：

- 调用者负责与实参相关的工作；
- 被调用者负责与自身局部环境相关的工作。

11.（参数传递机制，综合题）过程调用中的参数通常如何传递？为什么参数区域通常放在活动记录的开始位置？

参数可以通过寄存器或栈传递。

为了支持参数数量或类型可变的过程，参数区域通常被放置在活动记录的开始位置。

这样，调用者只需知道参数的数量和类型即可传递参数，而被调用者可以在运行时根据约定的位置访问这些参数。

12.（堆管理定位题）什么是堆？为什么堆管理比栈管理更复杂？

堆是用于存储生命周期不确定的数据对象的存储区域。

堆中对象的创建和释放顺序通常是不可预测的。

与栈不同，堆不满足后进先出的访问模式，因此需要复杂的内存管理算法来处理空间分配、回收和合并，从而避免严重的内存碎片问题。

13.（内存碎片，必考概念）什么是内存碎片？为什么内存碎片会降低程序的空间利用率？

内存碎片是指堆中存在大量不连续的小空闲块，导致即使总空闲空间足够，也无法满足新的分配请求。内存碎片会降低空间利用率，使程序更早耗尽可用内存，是堆管理中的主要问题之一。

14.（堆分配策略，对比型）什么是 first-fit 和 best-fit 分配策略？它们各自的优缺点是什么？

first-fit 策略选择第一个足够大的空闲块进行分配，分配速度快，空间局部性较好；

best-fit 策略选择满足请求的最小空闲块，能够提高空间利用率，但分配时间较长。

实际系统中常通过结合多种策略来权衡时间效率和空间效率。

15.（局部性与存储层次，老师很爱）什么是时间局部性和空间局部性？它们对程序性能有什么影响？

时间局部性指程序近期访问过的内存位置很可能在短时间内再次被访问；

空间局部性指与被访问位置相邻的内存位置也可能被访问。

局部性使得现代计算机能够通过存储层次结构显著降低平均内存访问时间，从而提高程序性能。

16.（存储层次结构，综合理解）为什么现代计算机采用存储层次结构？编译器在其中能发挥什么作用？

由于硬件限制，无法构造既大又快的存储器，因此现代计算机采用由寄存器、高速缓存、主存和虚拟存

储组成的存储层次结构。

编译器可以通过优化代码布局和数据布局，提高程序的局部性，从而更好地利用存储层次结构，提高程序的执行效率。

17.（总结型大题）请从“存储分配策略”和“过程调用支持”两个角度，总结运行时环境在编译器中的作用。

运行时环境通过合理的存储分配策略，为不同生命周期的数据对象提供合适的存储位置；

通过运行时栈和活动记录，支持过程调用、参数传递和返回。

二者共同保证了源语言中抽象概念在目标机器上的正确、高效实现。

第七章 Code Generation（代码生成）

1.（整体定位 + 难点 + 目标）什么是代码生成？代码生成器在编译器中的输入和输出分别是什么？为什么代码生成是一个困难的问题？

代码生成是编译器后端的核心阶段，其主要任务是将前端产生的中间表示（IR）和符号表信息翻译成语义等价的目标机器代码。

代码生成器的输入是中间表示以及符号表中记录的类型、存储位置等信息，输出是可以在目标机器上执行的目标程序（通常是汇编代码或机器代码）。

代码生成是一个困难的问题，因为从理论上讲，为给定源程序生成最优目标代码的问题是不可判定的；同时，代码生成中涉及的多个子问题（如寄存器分配）具有很高的计算复杂性。因此，实际编译器通常采用启发式方法，在保证正确性的前提下生成质量较高但不一定最优的代码。

2.（代码生成器三大任务，必考）代码生成器通常需要完成哪三个主要任务？请分别说明它们的作用。

代码生成器通常需要完成以下三个主要任务：

第一，**指令选择**，即为中间表示中的每一条操作选择合适的目标机器指令来实现；

第二，**寄存器分配与指派**，即决定哪些值保存在寄存器中，以及具体使用哪个寄存器；

第三，**指令调度（排序）**，即决定目标指令的执行顺序，以提高流水线效率或减少延迟。

这三项任务共同决定了目标代码的质量。

3.（目标机器模型，对比型）目标机器的指令集体系结构如何影响代码生成器的设计？请比较 RISC、CISC 和基于栈的机器。

目标机器的指令集体系结构对代码生成器的设计具有决定性影响。

RISC 机器通常具有大量寄存器、简单指令和简单寻址模式，适合生成结构清晰、规则性强的代码；

CISC 机器寄存器较少、指令复杂、寻址模式多样，代码生成器需要充分利用复杂指令以减少指令数量；

基于栈的机器通过操作栈顶元素完成计算，指令简单但交换操作频繁，效率较低。

不同体系结构要求代码生成器在指令选择和寄存器使用上采用不同策略。

4.（IR 与代码生成关系）为什么代码生成器通常假设输入的中间代码已经完成语法和语义检查？这样做有什么好处？

代码生成器通常假设输入的中间代码已经排除了语法错误和静态语义错误，并且必要的类型转换已经插入。

这样做的好处是简化代码生成器的设计，使其可以专注于生成高效的目标代码，而无需处理类型不匹配或非法操作等问题。同时，这种分工也体现了前端与后端职责分离的设计思想。

5.（基本块，引入动机）什么是基本块？基本块需要满足哪些性质？为什么基本块是代码生成和优化的基本单位？

基本块是一段连续的三地址指令序列，具有以下性质：

控制流只能从基本块的第一条指令进入；

除最后一条指令外，控制流不会在块中发生跳转或停止。

基本块内部的控制流是顺序的，使得编译器可以在不考虑复杂控制转移的情况下进行局部分析，因此基本块是局部代码生成和局部优化的基本单位。

6.（流图，结构题）什么是流图？流图中的结点和边分别表示什么？流图在代码生成中起什么作用？

流图是以基本块为结点、以控制流转移为边的有向图。

结点表示基本块，边表示程序执行过程中可能发生的控制转移关系。

流图为分析程序的控制结构提供了统一表示，是进行活跃性分析、循环识别和全局优化的重要基础。

7.（活跃性，老师很爱）什么是变量的活跃性？为什么活跃性信息对寄存器分配至关重要？

如果一个变量在某条指令之后仍可能被使用，则称该变量在该点是活跃的。

活跃性信息对寄存器分配至关重要，因为如果一个变量在后续不再被使用，其占用的寄存器可以被安全地重用；反之，如果错误地覆盖活跃变量的值，将导致生成错误的目标代码。

8.（下一次引用，过程理解题）什么是变量的下一次引用信息？它通常如何计算？

变量的下一次引用信息指的是某个变量在当前指令之后下一次被使用的位置。

在一个基本块中，下一次引用信息通常通过对三地址指令序列进行反向扫描来计算，并结合符号表记录变量的活跃性和使用位置。这些信息在局部代码生成和寄存器选择中具有重要作用。

9.（寄存器与地址描述符，必考）什么是寄存器描述符？什么是地址描述符？它们各自记录什么信息？

寄存器描述符用于记录当前某个寄存器中存放了哪些变量的值。

地址描述符用于记录某个变量的当前值可以在哪些位置找到，例如寄存器或内存。

这两类描述符为代码生成器提供了寄存器与变量之间的映射关系，是避免冗余加载和存储指令的关键数据结构。

10.（局部代码生成，过程型）在基本块内生成代码时，代码生成器的基本目标是什么？通常采用什么策略？

在基本块内生成代码的基本目标是尽量避免不必要的加载和存储指令，最大化寄存器的利用率。

常见策略包括：

优先使用已经包含所需值的寄存器；

在寄存器不足时，选择可以安全溢出的寄存器；

在基本块结束时，仅对出口处活跃的变量生成存储指令。

11.（溢出，概念题）什么是寄存器溢出？在什么情况下需要发生溢出？

寄存器溢出是指当没有空闲寄存器可用时，将某个寄存器中的值保存到内存中，以释放该寄存器。

当寄存器数量不足以容纳当前需要同时保留的活跃变量时，必须发生溢出。是否溢出以及溢出哪个变量，直接影响生成代码的效率。

12.（窥孔优化，老师爱考）什么是窥孔优化？窥孔优化的基本思想和特点是什么？

窥孔优化是一种局部优化技术，通过在目标代码或中间代码上使用一个小的滑动窗口，识别并替换低效的指令序列。

窥孔优化的特点是实现简单、效果明显，且一次优化可能产生新的优化机会，因此通常需要多次扫描代码。

13.（典型窥孔优化）请列举几类常见的窥孔优化，并说明其优化目标。

常见的窥孔优化包括：

冗余指令消除（如多余的加载和存储）；

不可达代码消除；

跳转链和无用跳转消除；

代数简化和常量传播。

这些优化旨在减少指令数量、降低执行时间。

14.（局部 vs 全局寄存器分配）局部寄存器分配和全局寄存器分配有何区别？各自适用于什么场景？

局部寄存器分配只在单个基本块内分配寄存器，实现简单，但在基本块边界需要频繁保存和恢复变量值。

全局寄存器分配在多个基本块甚至整个循环范围内保持寄存器分配的一致性，能够显著减少内存访问，但实现更加复杂。

15.（循环与寄存器，综合题）为什么循环在寄存器分配中具有特殊地位？全局寄存器分配通常如何利用循环信息？

由于程序的大部分执行时间通常花在循环中，因此将循环中频繁使用的变量保存在寄存器中可以显著提高性能。

全局寄存器分配通常优先为循环内部最活跃的变量分配固定寄存器，并在整个循环期间保持不变，从而减少重复加载和存储。

16.（正确性优先，老师视角）为什么说代码生成器最重要的设计目标是正确性？

代码生成器面对大量特殊情况，一旦生成错误代码，程序的语义将被破坏，后果严重。

因此，在保证正确性的前提下，才考虑代码质量、性能和实现复杂度，这是代码生成器设计中的首要原则。

17.（总结型大题）请从“效率”和“实现复杂度”两个角度，总结代码生成阶段在编译器中的设计取舍。

代码生成需要在生成高效目标代码和控制实现复杂度之间取得平衡。

通过启发式算法、分阶段设计和局部优化技术，编译器可以在保证正确性的前提下生成性能良好的代码，而不追求不可实现的最优解。

第八章 Machine-Independent Optimizations（机器无关优化）

1.（整体定位 + 目的）什么是代码优化？为什么说编译器进行的代码优化必须是“语义保持”的？

代码优化是指在不改变程序语义的前提下，对中间代码或目标代码进行改进，以提高程序运行效率或减少资源消耗的过程。

所谓“语义保持”，是指优化前后程序在所有可能的输入下都计算相同的结果。

这是因为编译器只负责提高程序性能，而不能改变程序的功能。如果优化改变了程序的计算结果，即使生成的代码运行更快，也是不正确的。

因此，代码优化的首要约束条件是保持程序语义不变。

2.（局部 vs 全局，老师很爱）什么是局部优化？什么是全局优化？二者的主要区别是什么？

局部优化是指仅在一个基本块内部进行的代码改进。

由于基本块内部不存在控制流分支，局部优化只需考虑顺序执行的语句，分析和实现相对简单。

全局优化是指跨越多个基本块、甚至整个过程范围内进行的优化。

全局优化需要考虑程序的控制流结构，通常依赖于数据流分析来判断优化是否安全。

二者的主要区别在于分析范围和所需信息的复杂程度。

3.（冗余的来源，理解型）为什么说程序中存在大量冗余操作？这些冗余通常从何而来？

程序中的冗余操作既可能来自程序员的编码习惯，也可能是高级语言抽象的副产品。

在高级语言中，数组访问、结构体字段访问等操作会在编译过程中被展开为一系列底层地址计算。这些底层计算往往在不同位置重复出现，但程序员并不直接看到，也无法手动消除。

通过让编译器在中间代码层次消除这些冗余，可以同时保证程序的可读性和运行效率。

4.（语义保持转换，必考）什么是语义保持转换？请列举几种常见的语义保持优化方法。

语义保持转换是指在不改变程序计算结果的前提下，对代码进行的等价变换。

常见的语义保持优化包括：

- 公共子表达式消除
- 复制传播
- 死代码消除
- 常量折叠
- 循环不变代码外提

这些转换的共同特点是：改变程序的执行方式，但不改变程序的功能。

5.（公共子表达式，老师最爱考）什么是公共子表达式？在什么条件下，一个表达式可以被安全地消除？

如果一个表达式在程序中被多次计算，并且在两次计算之间，其操作数的值没有发生改变，则该表达式称为公共子表达式。

只有在以下条件同时满足时，才能安全地消除公共子表达式：

- 表达式在之前已经被计算过；
- 表达式中涉及的变量在此期间没有被重新赋值；
- 表达式的计算不会产生副作用。

否则，消除该表达式可能会改变程序的语义。

6.（局部 vs 全局公共子表达式）局部公共子表达式消除和全局公共子表达式消除有何区别？为什么全局情况更复杂？

局部公共子表达式消除只在单个基本块内进行，只需保证在该基本块内变量值未发生变化。

全局公共子表达式消除需要考虑控制流路径，判断表达式在所有可能到达某个程序点的路径上是否具有相同的值。这通常需要数据流分析来支持。

由于存在分支和循环，全局公共子表达式消除比局部情况复杂得多。

7.（复制传播，承前启后）什么是复制传播？为什么复制传播常常作为其他优化的前置步骤？

复制传播是指在程序中出现形如 $x = y$ 的复制语句后，尽可能用 y 替代后续对 x 的使用。

复制传播本身不一定直接减少指令数量，但它可以暴露更多的优化机会，例如将某些赋值语句转化为死代码，从而为后续的死代码消除创造条件。

因此，复制传播通常与其他优化联合使用。

8.（死代码，必考定义）什么是死代码？死代码通常是如何产生的？

死代码是指其计算结果在程序的任何执行路径中都不会被使用的代码。

死代码往往不是程序员直接写出的，而是其他优化（如公共子表达式消除、复制传播）引入的中间结果。

通过消除死代码，可以减少不必要的计算，提高程序执行效率。

9.（循环优化，理解型）为什么循环是代码优化的重点区域？

在大多数程序中，循环体中的代码会被重复执行多次。

即使减少循环体中一条指令的执行次数，也可能对程序的整体运行时间产生显著影响。

因此，编译器在进行代码优化时，通常优先关注循环内部的代码。

10.（循环不变式，经典）什么是循环不变计算？为什么可以将循环不变计算移动到循环之前？

如果某个表达式在循环的每一次迭代中计算结果都相同，则称该表达式为循环不变计算。

由于该表达式的值在整个循环期间不发生变化，可以将其计算移动到进入循环之前，从而避免在每次迭代中重复计算，提高程序效率，同时不改变程序语义。

11.（数据流分析定位，必考）什么是数据流分析？为什么说大多数全局优化都依赖于数据流分析？

数据流分析是一类用于收集程序在各个程序点上可能出现的信息的算法。

全局优化需要判断某些性质在不同执行路径下是否成立，例如某个变量的定值是否到达某个程序点、某个表达式是否在所有路径上可用。这些问题都需要通过数据流分析来系统地解决。

12.（数据流抽象，理解型）为什么数据流分析必须对程序状态进行抽象？

由于程序中可能存在无限多条执行路径，且程序状态空间巨大，精确跟踪所有执行情况是不可行的。

数据流分析通过抽象程序状态，仅保留与分析目标相关的信息，从而在保证安全性的前提下，使分析在时间和空间上可行。

13.（到达定值，核心之一）什么是到达定值？到达定值信息在代码优化中有什么作用？

如果某个变量的定值能够沿着某条执行路径到达某个程序点，并且在此路径上没有被重新定值，则称该定值到达该程序点。

到达定值信息可以用于常量折叠、公共子表达式消除等优化，是最常用的数据流分析结果之一。

14.（活跃变量，和第七章呼应）什么是活跃变量？为什么活跃变量分析是逆向数据流问题？

如果一个变量在某个程序点上的值可能在之后的执行路径中被使用，则称该变量在该点是活跃的。

活跃变量分析需要从变量的“使用点”向前传播信息，因此其信息流方向与控制流方向相反，是一种逆向数据流问题。

15.（可用表达式，方向性理解）什么是可用表达式？为什么可用表达式分析使用交集作为交汇运算？

如果一个表达式在某个程序点之前已经被计算过，并且在所有到达该点的路径上，其操作数都没有被重新赋值，则称该表达式在该点是可用的。

由于必须保证表达式在所有路径上都可用，才能安全地复用其结果，因此可用表达式分析在路径汇合点使用交集作为交汇运算。

16.（三大数据流问题对比，老师级总结题）请比较到达定值、活跃变量和可用表达式这三种数据流分析在信息流方向和交汇运算上的差异。

到达定值是前向数据流问题，使用并集作为交汇运算；

活跃变量是逆向数据流问题，也使用并集作为交汇运算；

可用表达式是前向数据流问题，但使用交集作为交汇运算。

这些差异反映了不同分析目标对“安全性”和“精确性”的不同要求。

17.（保守性，老师视角）为什么说数据流分析结果通常是“保守的”？这种保守性有什么意义？

数据流分析通常假设程序中所有控制流路径都是可能执行的，即使在实际运行中某些路径永远不会发生。

这种保守性保证了编译器进行的优化不会改变程序语义。

即使因此错失某些优化机会，也比生成错误代码更加安全。

18.（总结型大题）请从“优化目标”和“分析基础”两个角度，总结机器无关代码优化的特点。

机器无关优化以提高程序性能为目标，同时不依赖具体目标机器结构。

其实现基础是对程序行为的静态分析，尤其是数据流分析。

通过在中间代码层次进行优化，可以为后续机器相关优化和代码生成奠定良好基础。

编译原理 · 跨章节概念题

1.（错误处理主线：词法 + 语法 + 语义）请从编译器整体角度说明错误处理为什么要“贯穿各阶段”。分别举例说明词法错误、语法错误、语义错误各自典型表现是什么；各阶段常见的错误恢复策略是什么；为什么错误恢复通常是“保守且局部”的？

错误处理贯穿编译全过程的原因在于：不同类型的错误出现在不同抽象层次，且后续阶段往往依赖前一阶段的输出，若不在合适阶段及时报告并尽量恢复，编译过程将无法继续，从而降低诊断信息的数量和质量。

- **词法错误**发生在字符级别，典型表现是出现无法匹配任何 token 模式的字符串（如非法字符、错误的数字格式）。词法阶段常用的恢复方式是**跳过若干字符直到能重新识别 token**，并记录行号/列号用于定位。
- **语法错误**发生在 token 序列层面，典型表现是 token 顺序不符合文法（如缺少分号、括号不匹配导致的结构错误）。语法阶段常用**恐慌模式恢复**：丢弃输入符号直到同步符号（如 `;`、`}`）出现；也可能采用**短语层次恢复**：尝试插入/删除少量 token 以继续分析。
- **语义错误**发生在含义层面，典型表现是类型不匹配、未声明即使用、错误的函数参数个数等。语义阶段通常采取**报告并尽可能继续**的策略，例如给出“错误类型”以避免连锁报错。

错误恢复之所以通常“保守且局部”，是因为编译器在静态分析阶段无法准确预测程序员的真实意图，过度“智能”的修复可能引入误导性的后续错误；局部修复能保证不破坏更多结构，从而输出更多可靠诊断信息。

📌 跨章节：第1章（贯穿）+ 第2章（词法错误）+ 第3章（语法错误恢复）+ 第4/5章（语义检查背景）

2.（从正则到 CFG：语言能力边界 + 工程分层）为什么编程语言的“词法”通常用正则表达式描述，而“语法”用上下文无关文法描述？请从表达能力、实现效率、以及工程分层三个角度说明这种选择的合理性，并举一个“正则做不了但 CFG 能做”的例子。

从表达能力看，正则表达式等价于有限自动机，只能描述正则语言，适合描述**局部、线性、无嵌套**的结构，如标识符、整数常量、关键字等；而程序语言的语法存在大量**递归与嵌套结构**（如括号匹配、语句块、表达式递归），需要 CFG 才能自然表达。

从实现效率看，正则模式可转化为 DFA，实现线性扫描、无需回溯，适合高速 tokenization；CFG 解析需要更复杂的栈结构与分析表（LL/LR），但能处理更强的结构约束。

从工程分层看，将词法与语法分离可以降低文法复杂度：语法规则只需面对 token，而无需处理空白、注释、数字格式等细节，从而提高可维护性和可移植性。

典型例子是匹配 $a^n b^n$ 这类需要计数关联的语言：正则表达式做不到，但 CFG 可以用递归产生式表达；在编程语言中，括号/块结构的正确匹配正是 CFG 的强项。

📌 跨章节：第2章（正则/DFA）+ 第3章（CFG/解析）+ 第1章（分层目的）

3.（短路求值主线：语义 + 控制流 + 回填）布尔表达式在编译时既可以“求值”也可以“控制跳转”。请说明短路求值的语义是什么；为什么短路求值更适合用跳转代码实现；在生成跳转代码时，回填技术解决了什么问题？请用 AND/OR 的控制流解释你的答案。

短路求值的语义是：对布尔表达式 $E1 \ \&\& \ E2$ ，若 $E1$ 为假则整体为假， $E2$ 不再求值；对 $E1 \ || \ E2$ ，若 $E1$ 为真则整体为真， $E2$ 不再求值。这一语义不仅影响性能，还影响正确性（因为 $E2$ 可能含副作用）。

短路求值更适合用跳转代码实现：

- 对 $E1 \ \&\& \ E2$ ：先为 $E1$ 生成“真跳到计算 $E2$ 、假跳到失败分支”的代码；只有当 $E1$ 为真时才进入 $E2$ 。
- 对 $E1 \ || \ E2$ ：先为 $E1$ 生成“真直接跳成功分支、假再去计算 $E2$ ”的代码。

生成跳转代码时常出现“目标标签尚未确定”的情况（例如 `if (E) S1 else S2` 中， E 的真假出口要跳到后面才出现的代码块）。回填技术通过先生成带空目标的跳转指令并记录其位置列表，待目标标签确定后统一填入，从而支持一趟式生成控制流代码。

📌 跨章节：第4章（语义动作/翻译方案）+ 第5章（布尔表达式翻译/回填）+ 第3章（控制结构依赖语法结构）

4.（从 AST 到三地址码：结构信息如何丢失与补偿）从语法树（或抽象语法树）到三地址码的翻译过程中，哪些“结构信息”被弱化或丢失？编译器通常如何通过基本块、流图等结构把控制流信息重新显式化？这种显式化对后续优化与代码生成有什么意义？

语法树/AST 是层次化结构，天然表达“嵌套与组合”，尤其适合语义分析与类型检查；但在翻译到三地址码时，复杂表达式被分解为线性的指令序列，原本的树形结构被“摊平”，控制结构也被展开为显式的跳

转与标签，很多高级结构信息（如循环结构的天然边界、表达式共享子树）不再直接可见。

为了重新显式表达控制流，编译器会把线性三地址码划分为**基本块**，并以基本块为节点构造**控制流图（流图）**。这样做使控制转移关系清晰可见，从而支持数据流分析（到达定值、活跃性、可用表达式等），并指导寄存器分配、指令选择以及分支优化。

显式化的意义在于：后续优化与代码生成本质上需要“知道哪里可能跳转、值在哪里可能被用到”，流图为此提供了统一框架。

📌 跨章节：第5章（AST→三地址码）+ 第7章（基本块/流图）+ 第8章（数据流分析依赖流图）

5.（活跃性与寄存器：为什么溢出不可避免）请从“活跃变量”的角度解释寄存器分配问题。为什么寄存器不足时必须发生溢出？在选择溢出对象时，为什么需要结合下一次引用信息？基本块边界为什么会使局部寄存器分配效果变差？

寄存器分配的核心是：尽量让“未来还会被使用”的值驻留在寄存器中以减少内存访问。活跃变量分析告诉我们：在某程序点之后变量是否可能再次被用到，若不再活跃，其值可以被覆盖而不影响语义。当同时活跃的变量数量超过可用寄存器数量时，无法让所有活跃值都驻留寄存器，因此必须把某些值存回内存以释放寄存器，这就是溢出。溢出选择会影响性能：若溢出的是很快要用到的变量，之后又要立刻加载回来，会产生额外开销。

因此，常结合**下一次引用**信息：优先溢出“下一次使用最晚或不再使用”的变量，以减少回装次数。

局部寄存器分配只在基本块内有效，跨基本块时由于控制流分支，通常需要在块末保存活跃变量、在块首恢复，导致频繁内存访问，使效果变差；这也是全局寄存器分配更有效但更复杂的原因。

📌 跨章节：第7章（活跃性/下一次引用/溢出）+ 第8章（活跃变量分析思想）+ 第6章（内存访问与运行时成本背景）

6.（过程调用与寄存器：调用约定的“为什么”）过程调用会破坏哪些运行时信息？为什么调用序列必须保存/恢复部分寄存器？请从“调用者保存”和“被调用者保存”的划分角度说明这种设计的目的，并解释它如何影响代码生成与寄存器分配。

过程调用会改变控制流并引入新的活动记录，从而可能破坏调用者当前的执行状态，包括：返回地址、部分寄存器中的临时值、栈指针/帧指针等。为了保证返回后调用者能继续正确执行，调用序列必须保存必要的机器状态并在返回序列中恢复。

“调用者保存/被调用者保存”的划分是为了在正确性和开销之间平衡：

- 调用者保存适用于那些调用者在调用后仍需要的临时值；调用点最清楚哪些值重要，因此由调用者选择性保存可以避免无谓保存。
- 被调用者保存适用于被调用过程会使用但必须保证对调用者不产生副作用的寄存器；这样调用者可假设这些寄存器在调用前后不变。

这种设计直接影响代码生成：编译器需要根据调用约定决定哪些寄存器在 call 前后必须被保护；也影响寄存器分配策略：跨调用点仍活跃的值往往需要避免放入会被破坏的寄存器，或在调用前后插入保存/恢复代码。

📌 跨章节：第6章（调用/返回/活动记录）+ 第7章（寄存器使用与代码生成）

7.（优化“为什么安全”：副作用与别名）为什么很多看似合理的优化在遇到指针/数组访问时会变得困难？请从“别名（aliasing）”和“副作用”的角度说明：编译器在进行公共子表达式消除、代码移动等优化

时需要额外满足哪些条件？

当程序中存在指针、数组下标或函数调用时，表达式的操作数可能通过不同名字引用同一内存位置，即出现别名问题；此外，某些操作可能具有副作用（例如通过指针写内存、函数调用修改全局状态）。

在公共子表达式消除中，只有当两次计算之间相关内存位置未被修改时才能复用结果。若存在可能写同一地址的语句（如 `*p = ...`），即使表面上变量没变，也可能改变表达式的值，从而使优化不安全。

在代码移动（如循环不变外提）中，除要求表达式值不变外，还要保证移动不会改变可观察行为，例如不能跨越可能抛异常或具有副作用的语句随意移动。

因此，编译器需要更保守的分析或额外的别名分析来保证优化正确。

📌 跨章节：第5章（内存访问形式）+ 第8章（优化正确性条件）+ 第6章（运行时副作用背景）

8.（把主线串起来的终极综合题）请用“从源程序到目标代码”的视角，串联说明：词法/语法分析提供了什么结构信息；语义分析如何补充类型与名字信息；中间代码如何显式表达计算与控制流；优化为什么依赖数据流分析；代码生成为什么必须考虑寄存器与调用约定。

词法分析把字符流转换为 token 序列，语法分析在 token 序列上构造语法结构，使编译器获得程序的层次与嵌套信息。语义分析在语法结构上补充类型、作用域、名字绑定等信息，保证程序含义符合语言规则，并为翻译提供必要约束。

中间代码生成把高级结构翻译为显式的计算与控制流形式（如三地址码、基本块与跳转），使后续阶段能够在统一表示上进行变换。优化依赖数据流分析，是因为跨基本块的安全变换需要判断不同路径上的性质（到达定值、活跃性、可用表达式等），必须通过保守静态分析获得依据。

代码生成最终将中间表示映射到目标指令，需要在有限寄存器条件下安排值的位置，并遵循过程调用约定保存/恢复必要状态，以保证跨调用点的正确执行。整个流程体现了“结构→语义→显式控制流→安全优化→机器约束映射”的逐层落实。

📌 跨章节：1–8章主线贯通